File 1 - `normalization/query_normalizer.py`

Change: Add this just above `DIRECT_CORRECTIONS`

```
BANK_ABBREVIATIONS = {
    "sbi": "STATE BANK OF INDIA",
    "pnb": "PUNJAB NATIONAL BANK",
    "bob": "BANK OF BARODA",
    "boi": "BANK OF INDIA",
    "hdfc": "HDFC BANK",
    "icici": "ICICI BANK LIMITED",
    "uco": "UCO BANK",
    "cbi": "CENTRAL BANK OF INDIA",
    "ubi": "UNION BANK OF INDIA",
}
```

Change: Inside `replace()` function, add bank block at the very top:

```
def replace(match: re.Match) -> str:
    token = match.group(0)
    lowered = token.lower()

    # Bank abbreviation expansion — add this
    if lowered in BANK_ABBREVIATIONS:
        replacement = BANK_ABBREVIATIONS[lowered]
        corrections[token] = replacement
        return replacement

    # rest of function unchanged ...
```

Change: Add two entries inside `DIRECT_CORRECTIONS`:

```
DIRECT_CORRECTIONS = {
    # ... existing entries ...
    "familys": "families",       # add this
    "famlies": "families",       # add this
}
```

File 2 - `retrieval/schema_retriever.py`

Change: Find and replace the trigger word line:

```
# BEFORE
if any(token in question.lower() for token in ["show", "list", "display",
"beneficiary", "all"]):

# AFTER
if any(token in question.lower() for token in ["show", "list", "display",
"beneficiary", "all", "who", "which", "has", "families", "most", "members",
"find", "give", "fetch", "get"]):
```

File 3 - `prompting/prompt_builder.py`

Change: Replace the entire `build()` method's column_lines loop:

```
# BEFORE
column_lines.append(
    f"- {column.qualified_name} (business meaning: {column.business_name};
"
    f"{column.data_type}, {indexed}): {column.description}; "
    f"aliases: {', '.join(column.aliases)}{sample_values}"
)

# AFTER
column_lines.append(
    f"- {column.qualified_name} ({column.data_type}, {indexed}): "
    f"{column.description}{sample_values}"
)
```

Change: Replace location_block assignment:

```
# BEFORE
location_block = (
    "\nLocation classification (use these exact conditions — do not
override them):\n"
    + "\n".join(location_rules)
) if location_rules else ""

# AFTER
location_block = (
    "\nLocation rules:\n" + "\n".join(location_rules)
) if location_rules else ""
```

Change: Replace location_rules append inside the for loop:

```
# BEFORE
if kind == "district":
    location_rules.append(
        f"- The location '{display_token}' IS one of the 41 Rajasthan
districts. "
        f"Filter using: family.district = '{canonical}'"
    )
else:
    location_rules.append(
        f"- The location '{display_token}' is NOT one of the 41 Rajasthan
districts. "
        f"You MUST filter using: (family.block LIKE '%{display_token}%' OR
"
        f"family.village LIKE '%{display_token}%'). "
        f"Do NOT use family.district for this location."
    )

# AFTER
if kind == "district":
    location_rules.append(
        f"- '{display_token}' is a district. Use: family.district =
'{canonical}'"
    )
else:
    location_rules.append(
        f"- '{display_token}' is NOT a district. Use: "
        f"(family.block LIKE '%{display_token}%' OR family.village LIKE
'%{display_token}%')"
    )
```

Change: Replace the `bank_details.bank_name` dynamic rule:

```
# BEFORE
if "bank_details.bank_name" in result.columns:
    dynamic_rules.append(
        "- bank_name: Always use UPPER(bank_name) LIKE
'%SEARCH_TERM_UPPER%'."
    )

# AFTER
if "bank_details.bank_name" in result.columns:
    dynamic_rules.append(
        "- bank_name: Always use UPPER(bank_name) LIKE
'%SEARCH_TERM_UPPER%'.\n"
        "  * If filtering by district too, JOIN family table."
    )
```

Change: Replace the `family.is_rural` dynamic rule:

```
# BEFORE
if "family.is_rural" in result.columns:
    dynamic_rules.append(
        "- is_rural: INTEGER — 1 = rural, 0 = urban."
    )

# AFTER
if "family.is_rural" in result.columns:
    dynamic_rules.append(
        "- is_rural: 1 = rural, 0 = urban. Use = not LIKE."
    )
```

Change: Replace the `caste_category` dynamic rule:

```
# BEFORE
if "member.caste_category" in result.columns:
    dynamic_rules.append(
        "- caste_category filtering: Use ONLY: 'SC', 'ST', 'OBC', 'GEN'.\n"
        "  * 'General category' means caste_category = 'GEN' (NOT
'General')."
    )

# AFTER
if "member.caste_category" in result.columns:
    dynamic_rules.append(
        "- caste_category: Use = 'SC', 'ST', 'OBC', or 'GEN'. Never use
LIKE."
    )
```

Change: Add new family member count block. Add this after the `is_rural` block:

```
if "family.family_id" in result.columns and "member.family_id" in
result.columns:
    dynamic_rules.append(
        "- member counts: GROUP BY family.family_id,
family.family_head_name "
        "then HAVING COUNT(*) > N. "
        "SELECT family.family_head_name — never family_id. "
        "Use COUNT(*) not COUNT(member.member_id)."
```

```
    )
```

Change: Critical bug fix - move `dynamic_rules_block` computation to AFTER all appends:

```
# BEFORE (wrong position — was computed before family block)
dynamic_rules_block = ("\n" + "\n".join(dynamic_rules)) if dynamic_rules
else ""

if "family.family_id" in result.columns and ...:
    dynamic_rules.append(...)   # this was AFTER the block was already
computed!

# AFTER (correct — compute only after ALL appends done)
if "family.family_id" in result.columns and ...:
    dynamic_rules.append(...)

# compute LAST
dynamic_rules_block = ("\n" + "\n".join(dynamic_rules)) if dynamic_rules
else ""
```

Change: Replace the entire return f-string prompt:

```
# BEFORE (original long prompt ~30 lines)
return f"""You are a SQL generator for a Jan Aadhaar-style relational
database.
Return SQL only. No markdown. No comments. No explanation.
... (long version)

# AFTER (lean version)
return f"""You are a SQL generator for a Jan Aadhaar relational database.
Return SQL only. No markdown. No explanation.
One read-only SELECT statement only.
Never use INSERT, UPDATE, DELETE, DROP, ALTER, CREATE, TRUNCATE, or any
DDL/DML.
Use only supplied tables and columns. Do not invent columns.
Avoid SELECT *. Never put family_id or bank_id in SELECT — use them only in
JOIN.
Generate {dialect_desc} SQL.
Name columns: always use LIKE '%value%', never exact =.
Gender: 'Male'/'Female'. Marital: 'Married'/'Unmarried'/'Widow'.
Caste category: 'SC'/'ST'/'OBC'/'GEN'. Senior citizen: age >= 60. Child:
age < 18.
Rural: is_rural = 1. Urban: is_rural = 0.
Always use = not LIKE for gender, caste_category, marital_status,
is_rural.{location_block}{dynamic_rules_block}

Tables: {', '.join(result.tables)}
Columns:
{chr(10).join(column_lines)}
Joins:
{chr(10).join(relationship_lines) if relationship_lines else "none"}
{error_block}Question: {result.question}
SQL:"""
```

File 4 - `app.py`

Change: Add two module-level dicts just after imports:

```
# Add these two lines after all imports, before _post_process_sql
_DISTRICTS_LOWER = {d.lower() for d in RAJASTHAN_DISTRICTS_41}
_DISTRICT_CANONICAL = {d.lower(): d for d in RAJASTHAN_DISTRICTS_41}
```

Change: Extend Step 2 - add LIKE without wildcards fix for bank_name:

```
# Add this block right after the existing LIKE '%val%' fix in Step 2
# Fix LIKE 'val' without wildcards
sql = re.sub(
    r"\b((?:\w+\.)?bank_name)\s*LIKE\s*'([^'%]+)'",
    lambda m: (
        m.group(0) if m.group(0).upper().startswith("UPPER(")
        else f"UPPER({m.group(1)}) LIKE '%{m.group(2).strip().upper()}%'"
    ),
    sql, flags=re.IGNORECASE,
)
```

Change: Add new Step 4 - fix LIKE on exact categorical columns:

Add this as a new block after the existing Step 3 cat_replacer:

```
# ── Step 4: Categorical LIKE → = (fixes gender/caste/marital LIKE bug) ──
sql = re.sub(
    r"\b((?:\w+\.)?gender)\s+LIKE\s+'%?(Male|Female)%?'",
    lambda m: f"{m.group(1)} = '{m.group(2)}'",
    sql, flags=re.IGNORECASE,
)
sql = re.sub(
    r"\b((?:\w+\.)?caste_category)\s+LIKE\s+'%?(SC|ST|OBC|GEN)%?'",
    lambda m: f"{m.group(1)} = '{m.group(2).upper()}'",
    sql, flags=re.IGNORECASE,
)
sql = re.sub(

r"\b((?:\w+\.)?marital_status)\s+LIKE\s+'%?(Married|Unmarried|Widow)%?'",
    lambda m: f"{m.group(1)} = '{m.group(2)}'",
    sql, flags=re.IGNORECASE,
)
```

Change: Fix Step 5 (is_rural) - extend to catch LIKE as well as =:

```
# BEFORE
sql = re.sub(
    r"\b((?:\w+\.)?is_rural)\s*=\s*['\"]?(\w+)['\"]?",
    rural_replacer, sql, flags=re.IGNORECASE,
)

# AFTER
sql = re.sub(
    r"\b((?:\w+\.)?is_rural)\s*(?:=|LIKE)\s*['\"]?(\w+)['\"]?",
    rural_replacer, sql, flags=re.IGNORECASE,
)
```

Change: Refactor Step 6 district replacer to use pre-built dict:

```
# BEFORE
def district_exact_replacer(match):
    col = match.group(1)
    val = match.group(2).strip().lower()
    for d in RAJASTHAN_DISTRICTS_41:        # loops list every call
        if val == d.lower():
            return f"{col} = '{d}'"
    return match.group(0)

# AFTER
def district_exact_replacer(match):
    col = match.group(1)
    val = match.group(2).strip().lower()
    canonical = _DISTRICT_CANONICAL.get(val)  # O(1) dict lookup
    if canonical:
        return f"{col} = '{canonical}'"
    return match.group(0)
```

Change: Add new Step 8 - fix district LIKE for known districts:

Add this as a new block after Step 7 (district = replacer) and before Step 9 (redirect):

```
# ── Step 8: District LIKE → = for known districts ──────────────────────
def district_like_replacer(match):
    col = match.group(1)
    val = match.group(2).strip()
    canonical = _DISTRICT_CANONICAL.get(val.lower())
    if canonical:
        return f"{col} = '{canonical}'"
    return match.group(0)

sql = re.sub(
    r"\b((?:\w+\.)?district)\s+LIKE\s+'%?([^'%]+?)%?'",
    district_like_replacer, sql, flags=re.IGNORECASE,
)
```

Change: Add Step 10 — COUNT(member.member_id) → COUNT(*):

```
# ── Step 10: COUNT(member.member_id) → COUNT(*) ────────────────────────
sql = re.sub(
    r"COUNT\s*\(\s*member\.member_id\s*\)",
    "COUNT(*)",
    sql, flags=re.IGNORECASE,
)
```

Change: Add Step 11 — fix broken AVG subquery alias:

```
# ── Step 11: Fix broken subquery alias T2.member_count → T1 ────────────
sql = re.sub(
    r"SELECT\s+AVG\s*\(\s*T2\.member_count\s*\)",
    "SELECT AVG(T1.member_count)",
    sql, flags=re.IGNORECASE,
)
```

Change: Add Step 12 — fix family_count alias:

```
# ── Step 12: Fix misleading family_count alias → member_count
──────────────
```

```
sql = re.sub(
    r"COUNT\(\*\)\s+AS\s+family_count",
    "COUNT(*) AS member_count",
    sql, flags=re.IGNORECASE,
)
```

Change: Add Step 13 — remove stray family_id:

```
# ── Step 13: Remove stray family_id when family_head_name present
─────────
if re.search(r"family_head_name", sql, re.IGNORECASE):
    sql = re.sub(
        r",\s*\w+\.family_id\b",
        "",
        sql, flags=re.IGNORECASE,
    )
```

Change: Add Step 14 — replace bare family_id in SELECT:

```
# ── Step 14: Replace bare family_id in SELECT with family_head_name
────────
sql = re.sub(
    r"\bSELECT\s+(COUNT\(\*\)\s+AS\s+\w+,\s*)(\w+)\.family_id\b",
    lambda m: f"SELECT {m.group(1)}{m.group(2)}.family_head_name",
    sql, flags=re.IGNORECASE,
)
sql = re.sub(
    r"\bSELECT\s+(\w+)\.family_id,\s*(COUNT\(\*\))",
    lambda m: f"SELECT {m.group(1)}.family_head_name, {m.group(2)}",
    sql, flags=re.IGNORECASE,
)
sql = re.sub(
    r"\bSELECT\s+DISTINCT\s+(\w+)\.family_id\b",
    lambda m: f"SELECT DISTINCT {m.group(1)}.family_head_name",
    sql, flags=re.IGNORECASE,
)
```

Change: Add Step 15 — remove display_column alias:

```
# ── Step 15: Remove unnecessary display_column alias
──────────────────────
sql = re.sub(
    r"\b(\w+\.family_head_name)\s+AS\s+display_column\b",
    r"\1",
    sql, flags=re.IGNORECASE,
)
```

File 5 - `database/models.py`

Change: Add two missing model classes at the bottom:

```
# Add these two classes — they exist in schema.sql but were missing from
models.py

class SchemeBenefit(Base):
    __tablename__ = "scheme_benefits"
```

```
    scheme_benefit_id: Mapped[int] = mapped_column(Integer,
primary_key=True)
    member_id: Mapped[int] = mapped_column(ForeignKey("member.member_id"),
index=True)
    scheme_name: Mapped[str] = mapped_column(String(120))
    bpl_status: Mapped[str | None] = mapped_column(String(16))
    apl_status: Mapped[str | None] = mapped_column(String(16))
    nfsa_status: Mapped[str | None] = mapped_column(String(24))
    pension_eligibility: Mapped[str | None] = mapped_column(String(24))
    beneficiary_status: Mapped[str | None] = mapped_column(String(24))
    benefit_amount: Mapped[int | None] = mapped_column(Integer)


class Verification(Base):
    __tablename__ = "verification"

    verification_id: Mapped[int] = mapped_column(Integer, primary_key=True)
    member_id: Mapped[int] = mapped_column(ForeignKey("member.member_id"),
                                           unique=True, index=True)
    ekyc_status: Mapped[str] = mapped_column(String(24))
    aadhaar_seeding_status: Mapped[str] = mapped_column(String(24))
    jan_aadhaar_status: Mapped[str] = mapped_column(String(24))
    last_updated_date: Mapped[Date | None] = mapped_column(Date)
```

Change: Add default to `BankDetails.dbt_status`:

```
# BEFORE
dbt_status: Mapped[str] = mapped_column(String(24))

# AFTER
dbt_status: Mapped[str] = mapped_column(String(24), default="Active")
```

File 6 - `database/sample_data.py`

Change: Replace entire file contents

```
Original file was a copy-paste of query_results.py containing
execute_select_preview() — completely wrong file.

Replace entirely with seed_demo_data() function containing:
- 3 family records (Jaipur/Jodhpur/Bikaner)
- 6 member records
- 3 bank records
- 2 scheme_benefit records
- 2 verification records
```